Natural Language Decomposition and Interpretation of Complex Utterances

Harsh Jhamtani **Hao Fang** **Patrick Xia** **Eran Levy**
Jacob Andreas **Ben Van Durme**
Microsoft Semantic Machines <sminfo@microsoft.com>

arXiv:2305.08677v2 [cs.CL] 8 Jan 2024

Abstract

Designing natural language interfaces has historically required collecting supervised data to translate user requests into carefully designed intent representations. This requires enumerating and labeling a long tail of user requests, which is challenging. At the same time, large language models (LLMs) encode knowledge about goals and plans that can help conversational assistants interpret user requests requiring numerous steps to complete. We introduce an approach to handle complex-intent-bearing utterances from a user via a process of hierarchical natural language decomposition and interpretation. Our approach uses a pre-trained language model to decompose a complex utterance into a sequence of simpler natural language steps and interprets each step using the language-to-program model designed for the interface. To test our approach, we collect and release *DeCU*—a new NL-to-program benchmark to evaluate **De**composition of **C**omplex **U**tterances.[1] Experiments show that the proposed approach enables the interpretation of complex utterances with almost no complex training data, while outperforming standard few-shot prompting approaches.

1 Introduction

Neural sequence models, pre-trained on large datasets of language and code, are extremely effective at parsing natural commands into programs, database queries, and other structured representations of user intent (Chen et al., 2021; Li et al., 2021; Shin et al., 2021; Roy et al., 2022). However, developing an interface that enables a user to interact with a new API or software system still requires substantial system-specific data collection. Users, meanwhile, may not be aware of the scope of this data collection, and pursue an open-ended set of goals – including goals more complicated than those anticipated by system designers.

In this paper, we present DECINT, an approach to **dec**ompose complex utterances into a sequence of simpler NL steps, each resembling a simpler elementary utterance that an existing language-to-program **int**erpreter for the NL interface can parse to a sub-program. Consider the utterance *"Exchange the timing of my meetings with Jane and Smith"* (Figure 1). DECINT breaks the utterance down into four NL steps, using a pre-trained LLM and just a few annotated decompositions. The generated NL steps are parsed into programs, relying primarily on a relevant (to the step being parsed) subset of a larger set of existing elementary utterances associated with simpler programs in the target representation. DECINT thus enables an NL interface system to handle user requests representing complex goals (never seen by a semantic parser) by breaking them into a series of NL steps that are interpreted into APIs (never seen by an LLM). Our work is related to recent work which demonstrates that large language models (LLMs) encode knowledge that can be used to interpret complex user goals requiring numerous steps to complete, in setups such as question answering (Wolfson et al., 2020; Khot et al., 2022) and embodied agents (Ahn et al., 2022; Huang et al., 2022). Compared to such past work, we are concerned with generating programs in a carefully designed intent representation. Starting with labeled elementary utterances, we wish to be able to parse complex utterances that are broader in scope compared to the abundant elementary utterances.

To study utterance decomposition in NL-to-program space, we collect and release *DeCU*—a new benchmark dataset to evaluate models for **De**composition of **C**omplex **U**tterance. *DeCU* consists of (1) a set of elementary utterances and corresponding programs for managing calendar events and emails and (2) a diverse set of complex user utterances annotated with decompositions into sequences of elementary utterances and their cor-

[1] Our code and *DeCU* dataset will be released.

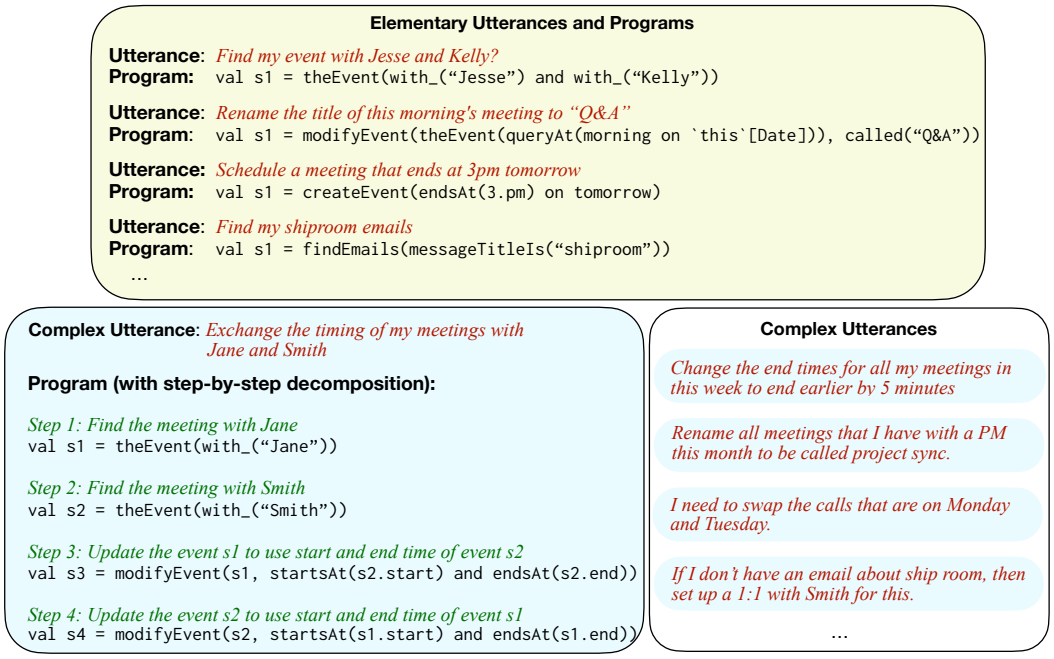

Figure 1: Parsing NL user utterances into programs. We study a scenario in which a large number of elementary utterances have been annotated with programs (top block), and we wish to build a model that can generalize to complex utterances (bottom blocks) requiring more elaborate programs. We introduce a method called DECINT that uses an LLM to decompose a *complex utterance* by predicting *simpler NL steps*, each of which is parsed to a `program` according to the annotated elementary utterances.

responding program fragments. Experiments on *DeCU* show that DECINT outperforms direct few-shot prompting approaches, making it possible to build NL interfaces that accomplish complex goals without large amounts of complex labeled data.

2 Task Overview

We study the problem of parsing an NL user utterance x into a program y that correctly reflects user intent (Figure 1). We focus on a version of the problem with the following characteristics:

- A domain developer has already collected a dataset of **elementary utterances** annotated with corresponding programs. These utterances represent narrow user goals associated with simple and short programs.
- At test time, the system must interpret **complex utterances**. Such utterances require longer programs representing much broader user goals.
- For a small number of complex utterances, we have access to annotations consisting of both natural language decompositions into elementary utterances, and program annotations for elementary utterances.

Annotated complex utterances will in general cover only a small part of the space of possible user requests, and our goal is to build a language-to-program model that can generalize to requests of very different kinds (Figure 1).

3 Data

Many existing relevant decomposition datasets focus on open-ended QA (Wolfson et al., 2020; Khot et al., 2021, 2022) or robotics domains with a relatively small number of fixed allowed actions (Puig et al., 2018; Shridhar et al., 2020). By contrast, we are interested in the task of parsing a user utterance to a program that represents the actions to be taken by the interface, grounded on a large number of fixed APIs. Moreover, we want to study how complex user utterances can be supported by the NL interface, without collecting a large amount of additional labeled data, by using decomposition in NL space. To study such multi-step complex intent decomposition, we introduce a new dataset we call *DeCU* (**De**composition of **C**omplex **U**tterances).

The utterances in *DeCU* focus on calendar events and emails. The dataset contains both elementary utterances and complex utterances. Elementary utterances (§3.2) are paired with declar-

Utterance 1: *Change my meetings with Abby and those with Dan this week to start 5 minutes later.*

Decomposition:

```
Step 1: Find events with Abby this week
val s1 = findEvents(with_("Abby") and queryAt(`this`[Interval[Date]] and isWeek))
Step 2: Find events with Dan and without Abby this week
val s2 = findEvents(with_("Dan") and not(with_("Abby")) and
    queryAt(`this`[Interval[Date]] and isWeek))
Step 3: Set all meetings from the list of events s1 to start 5 minutes later
val s3 = s1.map((x: Event) => modifyEvent(x, startsAt(x.start.local.time +
    5.minutes)))
Step 4: Set all meetings from the list of events s2 to start 5 minutes later
val s4 = s2.map((x: Event) => modifyEvent(x, startsAt(x.start.local.time +
    5.minutes)))
```

Utterance 2: *Decline any meeting invitations that are scheduled during my weekly team meeting.*

Decomposition:

```
Step 1: Find the event called "team meeting" that recurs weekly.
val s1 = theEvent(called("team meeting") and recurringWeekly)
Step 2: Find all events.
val s2 = findEvents()
Step 3: Filter events from list s2 to only include ones that intersect with event s1
    that are not s1.
val s3 = s2.filter((x: Event) => x.interval.intersects(s1.interval) && x.id != s1.id)
Step 4: Decline events in the list s3.
val s4 = s3.map((x: Event) => respond(x, ResponseStatusType.declined))
```

Figure 2: Examples of complex utterances in *DeCU*. Each utterance is accompanied by decompositions consisting of a sequence of NL steps and associated program fragments, annotated by domain experts.

ative Scala3 programs based on a domain library (§3.1) that admits a fixed set of APIs and specified types. Complex utterances (§3.3) are annotated with a corresponding sequence of elementary utterances, each paired with a program. Only a few of these complex utterances are included in the training set; they are mainly used to form a test set.

Figure 1 illustrates an example, *"Exchange the timing of my meetings with Jane and Smith"*. How such an utterance should be decomposed is domain-dependent: here, the calendar API does not provide a single endpoint that can swap pairs of meetings; instead, the system must search for the two meetings individually, then update each of their times. Figure 1 shows a possible decomposition into four steps. The first generated NL step, *"Find the meeting with Jane"*, is translated to a program fragment: `val s1 = theEvent(with_("Jane"))`. Individual steps typically represent easier-to-solve inputs for the NL-to-program parser that primarily relies on the annotated elementary utterances.

In addition to domain-specific knowledge of APIs, decomposition of complex utterances often relies on domain-general reasoning and common sense knowledge – for example, to avoid double-counting meetings that match two search results (Figure 2, utterance 1), or to recognize that meetings cannot conflict with themselves (utterance 2).

3.1 Domain Library

The domain library defines the set of types and functions available for program annotations. Types model objects such as `Person` and `Event`, whereas functions represent actions that can be taken by the agent, including high-level APIs (*e.g.,* `createEvent`, `findEmails`), low-level operations (*e.g.,* `min`, `+`), predicate constructors (*e.g.,* `called`, `startsAt`), etc. The domain library for *DeCU* is packaged as standard Scala source code, consisting of 33 types and over 200 functions.[2]

3.2 Elementary Utterances

DeCU contains 841 elementary utterances paired with programs. A few examples are shown in the top box in Figure 1. These utterances are *elementary* in that they represent narrow user goals such as creating or deleting a single meeting, which can typically be achieved using a single API. As such, they have relatively short programs, generally less than 5 tokens.[3] Examples are written and reviewed by domain experts who are familiar with the domain library (on account of their experience from working with a deployed system leveraging such a library) and annotation guidelines.

3.3 Complex Utterances

To study how *complex* utterances can be supported by an NL interface, we collect a diverse set of more involved user requests, and annotate these with

[2] Some built-in types (*e.g.,* `String`, `Boolean`), functions (*e.g.,* `map`), and control flow statements (*e.g.,* `if`) are not explicitly defined and counted. Appendix B provides more details.

[3] To compute this statistic, programs are split into tokens based on heuristics, treating API names, argument names, and values as individual tokens.

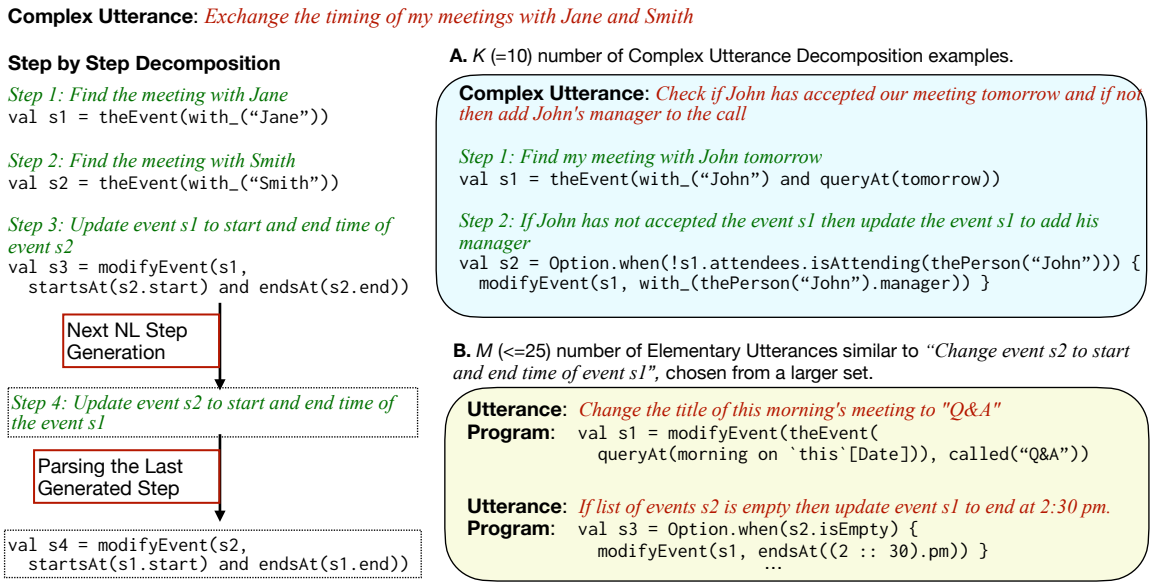

Figure 3: DECINT maps complex utterances into elementary steps, each of which is parsed in sequence to arrive at a final program. NL decomposition and program generation steps are interleaved. While parsing a step, up to M similar examples of elementary utterances are retrieved.

decompositions into elementary steps, along with programs for each step. As the name suggests, compared to elementary utterances, these utterances represent more complex and broader user goals, with the corresponding programs typically being much longer (an average of 14.5 tokens per program). To collect complex utterances, we employ a mix of manual authoring and automated utterance generation. Manual authoring is performed by domain experts with a focus on diversity and goals that require the composition of multiple calls to the domain APIs. For automated collection techniques, we generate utterances using GPT-3 (Brown et al., 2020) prompted with a few random examples of manually-authored utterances. About 60% of all the collected utterances were generated automatically. Appendix A provides more details on utterance collection. Examples are shown in Figure 1.

Decomposition Annotations: Six annotators familiar with the domain (annotators had past experience working with the domain library) decompose complex utterances into elementary ones. When results from earlier steps must be reused, these NL decompositions may include explicit reference to earlier step outputs (Figure 2). More information about annotator instruction is provided in Appendix A. Each annotation was additionally reviewed by two additional domain experts, separate from the set of 6 annotators.

Data Statistics: We collected a total of 210 unique complex utterances. The dataset is a mix of 126 utterances paired with annotated programs and 84 that are unannotated. As discussed later, in addition to reference-based metrics, we also provide various reference-less metrics that do not require annotations. While it is a relatively small count, note that most of the data (200 out of 210) is used to construct an evaluation set, as we are interested in learning to generalize from very small numbers of training examples. Annotated complex utterances in our full dataset exhibit a diverse range of properties (an utterance can have multiple): 55% use a map operation (for-loop), 36% contain actions based on a condition, 31% use a filter operation, 24% query about calendar/email, 37% contain a create meeting action, 9% contain a delete meeting action, and 31% contain a modify meeting action. The average number of decomposition steps in our data is 3, with a maximum of 7 steps. The average number of tokens in each program is 14.5[4], while the average number of tokens in the program fragment corresponding to a single step is 4.8. For comparison, the average number of tokens in the programs for elementary utterances is 4.5.

4 Approach

The DECINT approach, illustrated in Figure 3, maps a complex utterance x to a sequence of inter-

pretable lower-level NL steps ($z_1, z_2, ..$) that resemble elementary utterances. Each step or low-level utterance z_j is parsed into a program fragment y_j. In particular, DECINT maps from commands to programs according to the following iterative generative process:

1. **Natural Language Decomposition**: $z_j \sim p(\cdot \mid x, z_{<j}, y_{<j})$.
2. **Program Generation**: $y_j \sim p(\cdot \mid x, z_{\leq j}, y_{<j})$.

NL Decomposition (§4.1) and program generation (§4.2) steps are interleaved, with later portions of the language decomposition conditioned on earlier program fragments. In principle, one could also condition on the return values of the earlier program fragments (see Limitations section). We do not do so in this paper, as running the programs would require API implementations and input data.

4.1 Natural Language Decomposition

The NL decomposition stage generates the next NL step z_j conditioned on the user utterance x and any previously generated steps and program fragments. We obtain z_j by greedy decoding from a pre-trained LLM in a few-shot in-context learning setup (Brown et al., 2020). The model is prompted with $K = 10$ example decompositions, each of which consists of an utterance x followed by any previous steps and their program fragments, all concatenated together ($x, z_1, y_1, z_2, , ..., z_N, y_N$). We additionally found it useful to include a list of up to M elementary NL utterances at the start of the prompt (before the K decomposition examples), selecting the ones with highest BM25 similarity to the input utterance. This is intended to inform the model about the kind of elementary steps the NL-to-program parser can handle. (An example constructed prompt is shown in Appendix C.1.) Example decompositions are taken from the set of 10 complex utterances in the training split of *DeCU*.

DECINT's ability to perform NL decomposition thus results from a combination of parametric knowledge about the structure of programs in general (the result of pretraining) and non-parametric knowledge about the domain of interest (obtained via in-context learning). Together, these enable generalization to structurally novel user requests. For example, there are no training examples that involve exchanging the timing of two meetings (the test example in Figure 3), but DECINT nonetheless synthesizes a correct program.

4.2 Program Generation

The program generation step synthesizes a program fragment y_j for a given NL step z_j, conditioned on any preceding steps and incomplete program. This is a well-studied semantic parsing problem, and we design the NL-to-program parser largely following past work that applies pre-trained LLMs. We use in-context learning with dynamically selected prompt examples from the set of elementary examples data (Brown et al., 2020). As before, we use greedy decoding. Generated program fragments may refer to previously generated fragments using named step variables. For a given NL utterance or step, we identify up to M examples from the set of elementary utterances, where each example is an (utterance, program) pair (as shown in box B in Figure 3). The selection of the examples is based on the similarity of the utterance to the intermediate NL step being parsed. To compute similarity, we again use BM25, as in past work (Rubin et al., 2022; Roy et al., 2022). In pilot experiments on training data, we discovered it was useful to also include the K decomposition examples at the bottom of the prompt (detailed prompt example shown in Appendix C.1). This may be because the decomposition examples provide a demonstration of how to generate program fragments for a step conditioned on previous steps and help bridge any possible domain shift from elementary to complex utterances.

4.3 Baselines

The DECINT method decomposes a complex utterance into NL steps, separately parsing each step, and using internal variable references to assemble a larger program. The standard few-shot prompting approach for tasks like this one (*e.g.,* Roy et al., 2022) instead directly predicts the parse without generating the intermediate NL steps. We compare to this approach, which we denote **DIRECT-PRED**, in our experiments. There are a few key differences compared to the DECINT method. Complex utterance examples are presented without the intermediate NL steps (*i.e.,* each utterance is paired with a multi-line program). The output generation is a single-step process since there are no intermediate NL steps that need to be generated. As with DECINT, examples of elementary utterances are also included in the prompt. We also consider a **COT** (Wei et al., 2022) baseline, wherein the model

first predicts all intermediate NL steps and then predicts the program. Accordingly, the complex utterance examples in the prompt are annotated with intermediate steps. This baseline resembles the method proposed in Jiang et al. (2023). Note that compared to CoT, DecInt interleaves step generation and parsing, and dynamically updates the subset of exemplars from elementary utterances to be relevant to the step being parsed.

We also report results using a variant of DecInt that relies only on K decomposition exemplars but without access to elementary utterances (M=0 instead of 25). We refer to such a baseline as **Few-Shot**. Finally, we also report results for a variant of DecInt that uses only a single decomposition exemplar (K=1 instead of 10), and thus relies almost entirely on the elementary utterances from the underlying domain. We refer to the variant as **Elementary-Only**.

5 Experiments

5.1 Evaluation

Overlap with Reference Programs: We report Exact Match (**EM**) and character-based edit distance (**CER**) metrics[5] against the gold program. Before computing these metrics, we normalize the programs by lowercasing the entire program and removing extra spaces. Since there can be multiple possible ways to express the target multi-line program, Exact Match can only be viewed as a lower-bound metric for this task. These metrics are reported only for the subset of the data that consists of annotated reference programs.

Well-formed Evaluation: Additionally, we report the fraction of predictions that are valid (**WellForm**) under the domain library, *i.e.,* the full program follows correct syntax and only uses functions available in the library. Note that WellForm does not necessarily represent correctness with respect to the user goal. We report the metric for the entire test set.

Program Correctness: Finally, we report the overall correctness of the generated programs. We define a program to be correct overall if: it is well-formed, and correctly represents the user request. We use GPT-4 (gpt-4-32k) (OpenAI, 2023) to rate the correctness of the generated programs (**Correct**). The prompt consists of an instruction and four manually labeled exemplars (two "correct" and two "incorrect") followed by the test example. Each example is a user utterance followed by the associated program. The label is a natural language caption/explanation of the generated program, followed by a final verdict on whether either the generated program is "correct" or "incorrect" for the given user utterance – following a chain-of-thought style prediction[6]. Since we have an automatic static analysis to infer exactly which programs are well-formed (WellForm), outputs that are not well-formed are automatically considered to be incorrect as per the definition above (but are included in the denominator for all evaluations). Note that the Correct metric is reference-less, is easier to scale than human evaluations, and correlates well with human ratings (Section 5.3).

[5]https://huggingface.co/spaces/evaluate-metric/cer

System	Correct↑	WellForm↑	EM↑	CER↓
Direct-Pred	0.34	0.36	0.04	0.44
CoT	0.25	0.29	0.05	0.46
Few-Shot	0.13	0.19	0.00	0.50
Elementary-Only	0.23	0.31	0.04	0.54
DecInt	**0.41**	**0.46**	0.05	**0.40**

Table 1: Quality of the generated program for complex utterances under various automated metrics.

5.2 Setup

We consider the task of parsing complex utterances in *DeCU* given only ten complex utterances (annotated with decompositions) to be used as training data (exemplars for in-context learning). We report results on the test set consisting of the remaining 200 complex utterances. We use a maximum of $M \leq 25$ additional elementary utterances (as many as permitted by the LM's context window) selected according to BM25 similarity with the step being parsed. We use OpenAI's *text-davinci-003* model as the LLM for generating each NL step as well as for parsing it into a program.

5.3 Evaluation of Generated Program

Table 1 reports various automated metrics. DecInt outperforms all the baselines, sometimes by a wide margin. As can be seen in the table, DecInt outputs receive an overall correctness score (**Correct**) of 41% for complex utterances compared to 34% and 25% for the baselines Direct-Pred and CoT respectively.[7] We posit that DecInt is

[6]The exact prompts used in Correct are presented in Appendix D

[7]The differences are significant ($p < 0.05$) using bootstrap resampling.

Utterance 1: *Make me a meeting at 3 PM mountain time and refuse any meetings that happen at the same time.*

DECINT

```
Step 1: Find events at 3 pm mountain time
val s1 = findEvents(queryAt(3.pm inZone TimeZone("MST")))
Step 2: Refuse any events in the list s1
val s2 = s1.map(x => respond(x, withResponse(ResponseStatusType.declined)))
Step 3: Create a meeting at 3 pm mountain time
val s3 = createEvent(createAt(3.pm inZone TimeZone("MST")))
```

DIRECT-PRED

```
val s1 = createEvent(createAt(3.pm inZone TimeZone("MST")))
val s2 = findEvents(queryAt(3.pm inZone TimeZone("MST")))
val s3 = s2.map(x => refuseEvent(x))
```

COT

```
Step 1: Create a meeting at 3 pm mountain time
Step 2: Refuse any meetings that happen at the same time"}
val s1 = createEvent(createAt(3.pm inZone TimeZone(\"MST\")))
val s2 = findEvents(queryAt(3.pm inZone TimeZone(\"MST\"))).map(x => refuseEvent(x))
```

Comment We observe that hallucination of functions is much more prominent in the baselines DIRECT-PRED and COT, as can be seen in the use of the non-existing function `refuseEvent`

Utterance 2: *I'd like to divide the meeting at noon in two, so split off the last fifteen minutes as a separate event.*

DECINT

```
Step 1: Find the event at noon
val s1 = theEvent(createAt(noon))
Step 2: Subtract 15 minutes from the end time of the event s1
val s2 = s1.end - 15.minutes
Step 3: Update the event s1 to end at the time instant s2
val s3 = modifyEvent(s1, endsAt(s2))
Step 4: Create a new event starting at the time instant s2
val s4 = createEvent(startsAt(s2))
```

Comment Output program from DECINT was judged as incorrect since it doesn't specify the duration of the second event.

Figure 4: Outputs from DECINT and baselines on sample complex utterances in *DeCU*.

able to make more effective use of pretraining by breaking down a complex command into NL steps and retrieving relevant exemplars for each step. Further, FEW-SHOT, that is equivalent to DECINT with M=0, fares badly, suggesting that DECINT relies on information from elementary utterances in addition to supervised decompositions. Finally, ELEMENTARY-ONLY, which is equivalent to DECINT with K=1, also does worse than DECINT, suggesting the usefulness of a handful of supervised decompositions. Note, however, that a 54% of the predictions from DECINT are not well-formed, indicating that even structural generalization in *DeCU* remains a major challenge. Nonetheless, DECINT fares better compared to other methods on WellForm metric.

Human Evaluation for Program Correctness: We also obtained the overall program correctness rating ("correct" vs "incorrect" for a user utterance) from human evaluators familiar with the domain library. Just as was the case with **Correct** metric, outputs that are not well-formed are automatically considered incorrect. The aggregate scores for DECINT, DIRECT-PRED and COT (our method and the two top performing baselines as per automated **Correct** metric) under human evaluation are 41%, 33% and 26% respectively, which are very close to the scores for these methods under the automated **Correct** metric. Additionally, we observe a high correlation between human annotator-provided judgment and **Correct** judgments (a more detailed correlation analysis is provided in the Appendix D).

Results with other LLMs: We also report results using GPT-4 (*gpt-4-32k*) and LLAMA-2-70B (Touvron et al., 2023) as the underlying LLM. Due to cost considerations, we report results only for the top three methods from Table 1. We observe that DECINT outperforms the baselines, demonstrating that the proposed approach is effective across underlying LLMs (Table 2).

System	Correct↑/ WellForm↑ GPT-4	LLAMA2-70B
DIRECT-PRED	0.35 / 0.39	0.25 / 0.41
COT	0.37 / 0.40	0.23 / 0.32
DECINT	**0.49 / 0.56**	**0.35 / 0.50**

Table 2: Results using GPT-4/LLAMA-2-70B as the underlying LLM.

5.4 Evaluation of NL Decomposition

We measure whether the NL decomposition steps altogether are *sufficient and correct* to complete the user request.[8] For example, the output from DECINT for the second utterance in Figure 4 is *not* sufficient and correct because the fourth step fails to specify the duration of the meeting, which is supposed to be 15 minutes as per user request. A random subset of 40 of DECINT NL predictions and corresponding expert annotations were manually labeled by one of the authors as correct or incorrect. The expert annotations and DECINT predictions were rated as 98% and 85% correct, respectively. Future work can explore ways to further improve the accuracy of the predicted NL steps. We also conducted a step-level evaluation, which we discuss in Appendix D.

5.5 Qualitative Analysis

We provide example predictions in Figure 4, with additional examples provided in Figure A4 and Figure A6 in the Appendix. Additionally, we perform an error analysis of the NL-to-program step of DECINT. We restrict the study to the predictions that were labeled as incorrect in Table 1. The most common issues are those that make the program not well-formed, as summarized in Table 1. Many errors are due to *nonexistent APIs / API arguments* (21% of the incorrect programs have at least this problem) and *nonexistent type attribute* (43%). A smaller number result from even more basic *syntax errors* and *type mismatches* (17%). Future work could constrain the outputs of the parser (Shin et al., 2021) to only use allowed functions and follow correct syntax, though such approaches can substantially increase the cost of decoding.

A few errors result from predictions that capture only *partial user intent* (6%). For example, for utterance 2 in Figure 4, the prediction does not capture the user intent of creating the second event for 15 minutes. Many of the remaining errors involve more fundamental semantic mismatches between user intents and model outputs. For example, for *"Loop around all my 1/1 meetings this week so that they also happen next week"*, the prediction updates the meetings this week instead of creating another set of meetings next week.

[8] Unless stated otherwise, all analysis uses outputs with *text-davinci-003* as the underlying LLM

6 Related Work

Past work has explored using command decomposition to break down complex tasks or requests into smaller subtasks that are easier to manage. The LaMDA model (Thoppilan et al., 2022), for example, is capable of breaking down "How to" type queries into steps. However, generated steps are not tied to any actions or APIs, and are more in the form of a narrative rather than executable steps.

Khot et al. (2021) decompose a question into subquestions that can be answered by a neural factoid single-span QA model and a symbolic calculator. Drozdov et al. (2022) decompose an utterance using a syntactic parse. However, not all utterances in our dataset would lend to such a style of decomposition, since all required actions might not align to a part of the parse. Recent work (Jiang et al., 2023) has also explored first generating an entire plan in NL and then generating a program. Paranjape et al. (2023) focus on using tools and python scripts to complete a given task such 'Translate into Pig Latin'. Compared to such past work, the complex utterances in our case are decomposed into intermediate steps that are parsed into a sub-program in the target representation as opposed to generating Python programs. Additionally, these sub-programs are a part of the final program output and thus we care about the accuracy of intermediate steps as well.

A related area of research involves grounding high-level tasks, expressed in natural language, to a chosen set of actionable steps that a robot could take (Sharma et al., 2022; Singh et al., 2022; Ahn et al., 2022; Huang et al., 2022). Huang et al. (2022) propose a method to ground high-level tasks such as 'make breakfast' to a set of actionable steps such as 'open fridge'. Such work typically assumes a fixed inventory of low-level actions, and may not directly apply to setups like ours that additionally concerns with the interpretation of the steps into a rich target domain representation.

7 Conclusion

We have presented DECINT, an approach for interpreting complex user utterances by decomposing them into elementary natural language steps. To evaluate methods for generating programs from natural language requests, we have introduced the *DeCU* dataset, featuring a diverse set of utterances requiring substantial generalization from a small training set. Experiments on *DeCU* show that

DECINT outperforms a standard few-shot prompting approach to program generation, with additional analysis revealing opportunities for improvement in both natural language decomposition and program generation phases.

Limitations

The approach described in this paper does not condition on *execution* results from intermediate steps, only generated programs themselves. Incorporating execution would improve the potential expressiveness of the model (*e.g.,* by allowing it to implement control flow operations conditioned on program results or exceptions). Program results might themselves be natural language strings (e.g., reminders or search results), enabling future extensions of DECINT to support an even richer space of requests. We used pre-trained large-language models from OpenAI, through paid API access that may not be available for everyone or in the future. However, our experiments using LLAMA-2-70B model (Touvron et al., 2023) should be easily reproducible. We report and discuss several evaluation measures to check the quality of the predictions. One could also examine the outcome and side effects from executing the programs. However, lots of the queries require setting up a populated database and the outcome would vary as we execute the programs in different sandbox environments. Developing an evaluation setup with sandbox executions is challenging and remains an open research question.

Ethics Statement: We leverage pre-trained neural language models such as GPT-3, and systems built using our approach might inherit some biases present in these pre-trained models. We build a system for NL-to-program, that users can leverage to command various NL interfaces. Such systems are not perfectly accurate and should be carefully deployed since they may lead to unintended side effects.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, and Mengyuan Yan. 2022. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv: 2204.01691*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *arXiv: 2107.03374*.

Andrew Drozdov, Nathanael Schärli, Ekin Akyürek, Nathan Scales, Xinying Song, Xinyun Chen, Olivier Bousquet, and Denny Zhou. 2022. Compositional semantic parsing with large language models. *arXiv: 2209.15003*.

Olga Golovneva, Moya Peng Chen, Spencer Poff, Martin Corredor, Luke Zettlemoyer, Maryam Fazel-Zarandi, and Asli Celikyilmaz. 2023. ROSCOE: A suite of metrics for scoring step-by-step reasoning. In *Proceedings of the Eleventh International Conference on Learning Representations*.

Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9118–9147. PMLR.

Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning code generation with large language model. *arXiv: 2303.06689*.

Tushar Khot, Daniel Khashabi, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2021. Text modular networks: Learning to decompose tasks in the language of existing models. In *Proceedings of the 2021 Conference of the North American Chapter of the*

Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021, pages 1264–1279. Association for Computational Linguistics.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. *arXiv: 2210.02406*.

Haoran Li, Abhinav Arora, Shuohui Chen, Anchit Gupta, Sonal Gupta, and Yashar Mehdad. 2021. MTOP: A comprehensive multilingual task-oriented semantic parsing benchmark. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, EACL 2021, Online, April 19 - 23, 2021*, pages 2950–2962. Association for Computational Linguistics.

Jekaterina Novikova, Ondrej Dusek, and Verena Rieser. 2017. The E2E dataset: New challenges for end-to-end generation. In *Proceedings of the 18th Annual SIGdial Meeting on Discourse and Dialogue, Saarbrücken, Germany, August 15-17, 2017*, pages 201–206. Association for Computational Linguistics.

OpenAI. 2023. Gpt-4 technical report.

Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*.

Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. 2018. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8494–8502.

Subhro Roy, Sam Thomson, Tongfei Chen, Richard Shin, Adam Pauls, Jason Eisner, and Benjamin Van Durme. 2022. BenchCLAMP: A benchmark for evaluating language models on semantic parsing. *arXiv: 2206.10668*.

Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, pages 2655–2671. Association for Computational Linguistics.

Pratyusha Sharma, Antonio Torralba, and Jacob Andreas. 2022. Skill induction and planning with latent language. In *Proceedings of the Annual Association for Computational Linguistics*.

Richard Shin, Christopher H. Lin, Sam Thomson, Charles Chen, Subhro Roy, Emmanouil Antonios Platanios, Adam Pauls, Dan Klein, Jason Eisner, and Benjamin Van Durme. 2021. Constrained language models yield few-shot semantic parsers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 7699–7715. Association for Computational Linguistics.

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10740–10749.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2022. Progprompt: Generating situated robot task plans using large language models. *arXiv: 2209.11302*.

Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven Zheng, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Yanqi Zhou, Chung-Ching Chang, Igor Krivokon, Will Rusch, Marc Pickett, Kathleen S. Meier-Hellstern, Meredith Ringel Morris, Tulsee Doshi, Renelito Delos Santos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin Hoffman-John, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise Agüera y Arcas, Claire Cui, Marian Croak, Ed H. Chi, and Quoc Le. 2022. Lamda: Language models for dialog applications. *CoRR*, abs/2201.08239.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Weiyue Wang, Jan-Thorsten Peter, Hendrik Rosendahl, and Hermann Ney. 2016. CharacTer: Translation edit rate on character level. In *Proceedings of the First Conference on Machine Translation: Volume 2, Shared Task Papers*, pages 505–510, Berlin, Germany. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

Tomer Wolfson, Mor Geva, Ankit Gupta, Yoav Goldberg, Matt Gardner, Daniel Deutch, and Jonathan

Berant. 2020. Break it down: A question understanding benchmark. *Trans. Assoc. Comput. Linguistics*, 8:183–198.

A Data Collection

A.1 Complex Utterance Collection

The complex utterances in our data are collected by a mix of manual authoring and automated means described below:

Utterances Authored by Expert Annotators: A set of domain experts familiar with the elementary utterances are requested to author new complex utterances. They are informed of the following desiderata: 1) utterances should represent a more complex and broader intent compared to elementary utterances; 2) the set of utterances should be diverse. To encourage creativity and diversity, the annotators were prompted with a set of keywords, and they are asked to author an utterance that spans at least some of the provided keywords. Similar approaches have been found useful in past work (Novikova et al., 2017). The keywords are randomly sampled from a list of curated keywords, relevant to the calendaring and email domain. For each instance, we draw 5 keywords randomly from a much longer list of keywords constructed by the authors. Some of the keywords in our list are as follows: decline, pen, vacation, plan-my-day, project-sync, timezone, count-of-meetings, calendar-update, etc.

Automatically Generated Utterances : To scale the process of utterance collection and gather even more diverse utterances, we additionally generate complex utterances using GPT-3 (Brown et al., 2020), a pre-trained large language model. A few random examples of human-authored utterances are provided as in-context examples in the prompt, and new utterances are sampled. Specifically, we repeatedly sampled 10 utterances from the set of manually authored utterances to be used as prompt examples. We had additionally included an instruction 'Now generate more utterances that are different from the above ones'. We sampled a new utterance with a temperature of 0.8. About 60% of all the collected utterances were generated automatically via the described process.

Additional Information: Note that the utterances are limited to the English language, and expanding to other languages is a potential future extension. Additionally, note that the expert annotators were provided instructions that no personally identifiable information or offensive content should be present in the utterances. One of the authors also did a manual check of the collected data to ensure that the instructions were followed. All annotators were resident domain experts and were paid above the prevailing minimum page. The data annotators were provided with relevant information about the task and how the data would be used. Furthermore, the authors held an interactive session with the data annotators to give a brief overview and answer any questions.

A.2 Guidelines for Decomposition Annotations

Expert annotators familiar with the domain are requested to annotate the decompositions of the utterances. Figure A1 shows the interface used by the annotators. The interface automatically checks if the annotation is well-formed or not. Annotators are asked to skip any utterances that cannot be annotated as per the domain library. We had a total of 6 domain experts who annotated the data. To ensure high quality, each annotation was additionally reviewed by at least 2 domain experts, separate from the set of 6 annotators.

Annotators are given guidance that to the extent possible, each step in decomposition is supposed to resemble an elementary utterance. Additionally, to refer to results from earlier steps, the results from i^{th} can be referred to using variable s_i. An example can be seen in Figure A1. We additionally provide guidance that NL steps should be grammatically correct full sentences. Moreover, annotators are advised to write the imperative commands using the present tense verb (*e.g.,* prefer *"Filter reports . . . "* over *"Filtering reports . . . "*).

A.3 Decomposition Examples

Figure A2 shows some example decompositions of complex utterances in *DeCU*.

B Domain Library

The domain library defines the set of types and functions available in the domain. Types model the domain objects such as `Person`, `Event`, whereas functions represent actions that can be taken by the agent, including high-level APIs (*e.g.,* `createEvent`, `findEmails`), low-level operations (*e.g.,* `min`, `+`), predicate constructors (*e.g.,* `called`, `startsAt`), etc. The domain library is provided as Scala source code, so that dataset users can statically validate generated code by compiling

```yaml
dialogueId: d17
turns:
  - userUtterance: Add a meeting with Ryan and Smith on each day next week where I have fewer than 4 meetings.
    expressPlan: |-
      // step 1: Create a list to iterate through each day of the week
      val s1 = List[DayOfWeek](Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday)

      // step 2: Filter list s1 to include only the days where there are fewer than 4 meetings found on that day
      val s2 = s1.filter((x: DayOfWeek) => findEvents(queryAt(x /& (next[Interval[Date]] and isWeek))).results.size < 4L)

      // step 3: For each day in list s2 create a meeting with Ryan and Smith, on that day
      val s3 = s2.map((x: DayOfWeek) => createEvent(with_("Ryan") and with_("Smith") and createAt(x /& (next[Interval[Date]] and isWeek))))

```

Figure A1: The interface used by domain experts to annotate the decompositions. The interface points out the syntax, type, and missing function errors, enabling the experts to author well-formed annotations. These "express" annotations are subsequently normalized by stripping comments, removing type annotations, code formatting, and selecting canonical function names when there is ambiguity (`concat` vs. `+`). The resulting programs conform to Scala syntax.

it with the domain library code. Some builtin types (*e.g.,* String, Boolean), functions (*e.g.,* `map`, `Option.when`), control flow statements (*e.g.,* `if`) are not explicitly defined, but they can be used in the domain.

Natural language descriptions of entities or actions often omit some of their fields and focus on a subset of criteria that distinguish them from others. In *DeCU*, we represent these criteria as **predicates**, which are lambda functions that take one or more arguments and return a `Boolean` value. For example, a predicate that checks whether an event has the subject "planning" can be rewritten as `called("planning")`, where `called` is a predicate constructor defined in the domain library. These predicate constructors simplify the annotations, avoiding spelling out the details of the field comparisons. It also makes the program closer to natural language descriptions and potentially easier for LLMs to predict. To conjoin two predicates, the function `and` can be used.

Further, the domain library provides a collection of extension methods and implicit conversions which significantly simplify annotations for temporal expressions. For example, the function `on` below can be used to combine a time expression and a date expression of different types.

```
extension [T](time: T)(using Conversion[T,
    LocalDateTime => Boolean]) {

  def on[U](date: U)(using Conversion[U,
    LocalDateTime => Boolean]): LocalDateTime =>
    Boolean = ???

}
```

With this function and corresponding implicit conversions, "*3pm on Monday*" and "*morning on May 15*" can be consistently annotated as `3.pm on Monday` and `morning on (May /& 15)`, respectively, where `3.pm` returns `Time => Boolean`, `Monday` returns `DayOfWeek`, `morning` returns `Interval[Time]`, and `May /& 15` returns `Date => Boolean`.

C Additional Method Details

C.1 Prompt Example

Figure A3 shows a sample constructed prompt to generate a program fragment corresponding to the last generated step.

C.2 LLM APIs

We use OPEN-AI's APIs, as per their terms of use https://openai.com/policies/terms-of-use

D Additional Details on Experiments

Experiments with Elementary Utterances

To contextualize model performance on complex utterances, we conclude by analyzing how the NL-to-program semantic parser fares on elementary utterances in the DeCU dataset. We split the elementary utterances data, consisting of 841 utterances, into train, dev and test splits in the ratio 70:15:15. We manually tried a few tweaks (about 5 variations were tried) to the prompt structure and varied parameters such as the number of exemplars in the prompt, and picked the setup that resulted in

Utterance 1: *Change my meetings with Abby and those with Dan this week to start 5 minutes later.*

Decomposition:

```
Step 1: Find events with Abby this week
val s1 = findEvents(with_("Abby") and queryAt(`this`[Interval[Date]] and isWeek))
Step 2: Find events with Dan and without Abby this week
val s2 = findEvents(with_("Dan") and not(with_("Abby")) and
    queryAt(`this`[Interval[Date]] and isWeek))
Step 3: Set all meetings from the list of events s1 to start 5 minutes later
val s3 = s1.map((x: Event) => modifyEvent(x, startsAt(x.start.local.time +
    5.minutes)))
Step 4: Set all meetings from the list of events s2 to start 5 minutes later
val s4 = s2.map((x: Event) => modifyEvent(x, startsAt(x.start.local.time +
    5.minutes)))
```

Utterance 2: *For my reports who don't already have a 1/1 call with me this week, schedule a 1/1 with each one of them.*

Decomposition:

```
Step 1: Retrieve my direct reports
val s1 = me.directReports
Step 2: Filter out the reports that do not have a one-on-one meeting with me this week
val s2 = s1.filter((x: Person) => findEvents(with_(x) and isOneOnOne and
    queryAt(`this`[Interval[Date]] and isWeek)).results.isEmpty)
Step 3: Create a one-on-one meeting with those filtered reports in list s2
val s3 = s2.map((x: Person) => createEvent(with_(x) and isOneOnOne))
```

Utterance 3: *Decline any meeting invitations that are scheduled during my weekly team meeting.*

Decomposition:

```
Step 1: Find the event called "team meeting" that recurs weekly.
val s1 = theEvent(called("team meeting") and recurringWeekly)
Step 2: Find all events.
val s2 = findEvents()
Step 3: Filter events from list s2 to only include ones that intersect with event s1
    that are not s1.
val s3 = s2.filter((x: Event) => x.interval.intersects(s1.interval) && x.id != s1.id)
Step 4: Decline events in s3.
val s4 = s3.map((x: Event) => respond(x, ResponseStatusType.declined))
```

Utterance 4: *Find all the meetings this week where the duration is more than 2 hours and reschedule them to next week..*

Decomposition:

```
Step 1: Find all meetings happening this week
val s1 = findEvents(queryAt(`this`[Interval[Date]] and isWeek))
Step 2: Filter the list of events s1 to keep events with a duration greater than 2
    hours
val s2 = s1.filter(x => x.duration > 2.hours)
Step 3: Update each event in the list s2 to happen next week
val s3 = s2.map(x => modifyEvent(x, startsAt((next[Interval[Date]] and isWeek))))
```

Figure A2: Examples of complex utterances in *DeCU*. Each utterance is accompanied by decompositions consisting of a sequence of NL steps and associated program fragments, annotated by domain experts.

the highest exact match accuracy on the dev split. We report an exact match against the gold parse as has been used in the past work as well (Roy et al., 2022). Additionally, we note that there might be certain small deviations such as extra surrounding braces that do not invalidate the generated program, and exact match as a binary metric would penalize such deviations. So we additionally report a character-based edit distance measure (Wang et al., 2016) that might provide more fine-grained insights compared to binary exact match.

Method	Exact Match↑	Char Edit↓
LLM parser	0.60	0.10
Ablations:		
Using text-davinci-001	0.42	0.23
Random exemplars	0.21	0.41
Max 3 examples	0.60	0.13
Limit to 20% train data	0.33	0.26

Table A1: Evaluation results on parsing elementary utterances in the test set.

Results: Results on the parsing task for the test split are shown in Table A1. The parser, referred to as LLM Parser in the Table, gets 0.60 exact match and 0.10 character edit distance. We also consider a few variations of the parser:

(1) Using *text-davinci-001* instead of *text-davinci-003*: Leads to a large reduction in the automated metrics.

(2) Using random M examples in the prompt instead of dynamically selected ones based on similarity to test command: Performs the worst among all the variations, demonstrating that relevant example selection is very important.

(3) Reducing the number of in-context examples from $M = 20$ to $M = 3$: The metrics show little to

```
Step: Filter the list of events s2 to include only the events that are not the event s1
Program: val s3 = s2.filter(x => x.id != s1.id)
<EOS>

Step: Filter the list of events s3 to include only the events organized by me and which intersect with the event s1
Program: val s4 = s3.filter(x => x.organizerIs(me) && x.interval.intersects(s1.interval))
<EOS>

Step: Filter the list of persons s2 to include only people attending the event s1
Program: val s4 = s2.filter(x => s1.attendees.isAttending(x))
<EOS>

Step: Filter the list of persons s3 to include only the people attending the event s1
Program: val s5 = s3.filter(x => s1.attendees.isAttending(x))
<EOS>

Step: Filter the list of events s1 to only contain events with 1 other attendee
Program: val s = s1.filter(x => x.attendees.all.size == 2)
<EOS>

Step: Filter the list of days s1 to include only those days where there are at least 5 events
Program: val s = s1.filter(x => findEvents(queryAt(x /& (next[Interval[Date]] and isWeek))).size >= 5)
<EOS>

Step: Filter the list of events s5 to keep only those events that intersect with the interval s4
Program: val s6 = s5.filter(x => x.interval.intersects(s4))
<EOS>

Step: Filter the events in the list s1 to include only those where the other attendee has a job title of "PM"
Program: val s = s1.filter( x => getPersonFromAttendee(x.attendees.otherPeople.head).jobTitle == Some("PM"))
<EOS>

Step: Describe the number of events in the list s1
Program: val s2 = s1.size
<EOS>

Step: Update each event in the list s1 to last only for 30 minutes
Program: val s = s1.map(x => modifyEvent(x, lastsFor(30.minutes)))
<EOS>

Step: If the list s2 is empty then update the event s1 to end at 2:30 pm.
Program: val s3 = Option.when(s2.isEmpty){modifyEvent(s1, endsAt((2 :: 30).pm))}
<EOS>

Step: Filter the list of persons s1 to keep only those that don't have a one-on-one meeting with the user this week
Program: val s = s1.filter(x => findEvents(with_(x) and isOneOnOne and queryAt(`this`[Interval[Date]] and isWeek)).isEmpty)
<EOS>

Step: If there are more than 5 events in the list s1, decline the longest scheduled event
Program: val s = Option.when(s1.size > 5){respond(max(s1, (x => x.duration)), withResponse(ResponseStatusType.declined))}
<EOS>

Step: Change the event s1 to include Ben and remove Hao
Program: val s = modifyEvent(s1, with_("Ben") and not(with_("Hao")))
<EOS>

Step: Update the subject of each event in (list s1)[the list] to "1/1 with (corresponding person)"
Program: val s = s1.map(x => modifyEvent(x, called("1:1 with " + x.attendees.otherPeople.head.nameAndEmail.name.get)))
<EOS>

Step: Update each event in the list s2 to be called "project sync"
Program: val s = s2.map(x => modifyEvent(x, called("project sync")))
<EOS>

Step: Forward this email to the list of persons s2
Program: val s3 = forwardEmail(messageIdIs(theEmail(`this`[Email]).id) and messageWithTo(s2))
<EOS>

Step: Update each event in the list s3 to happen after the event s1
Program: val s5 = s4.map(x => modifyEvent(x, startsAt(after(s1.interval))))
<EOS>

Step: For each day in the list s2 create an event with the called "vacation time"
Program: val s = s2.map(x => createEvent(called("vacation time") and createAt(x /& (next[Interval[Date]] and isWeek))))
<EOS>

Step: Decline each meeting in the list s1
Program: val s = s1.map(x => respond(x, withResponse(ResponseStatusType.declined)))
<EOS>

Step: Check if the size of the list s1 is 5 or less
Program: val s = s1.size <= 5
<EOS>

Step: Update the event s1 to happen after 8am and before 5pm in the time zone s2.
Program: val s3 = modifyEvent(s1, startsAt(after(8.am) inZone s2) and endsAt(before(5.pm) inZone s2))
<EOS>

Step: Get the attendees of the event s1
Program: val s2 = s1.attendees.all
<EOS>

Step: Get the start time from event s1
Program: val s = s1.start.local.time
<EOS>

Step: Get the end time from event s1
Program: val s = s1.end.local.time
<EOS>


Utterance: Forward this email to all the attendees in the Standup event
Step 1: Find the event called Standup
Program 1: val s1 = theEvent(called("Standup"))
Step 2: Get the attendees of the event s1
Program 2: val s2 = s1.attendees.all
Step 3: Forward this email to the list of persons s2
Program 3: val s3 = forwardEmail(messageIdIs(theEmail(`this`[Email]).id) and messageWithTo(s2))
<EOS>

Utterance: If there is an "emergency review" meeting this week, then reschedule any events that happen 30 minutes before or after the meeting to next Friday.
Step 1: Find the event called "emergency review"
Program 1: val s1 = theEvent(called("emergency review"))
Step 2: Subtract 30 minutes from the start time of the event s1
Program 2: val s2 = s1.start - 30.minutes
Step 3: Add 30 minutes to the end time of the event s1
Program 3: val s3 = s1.end + 30.minutes
Step 4: Establish the time interval between the time instant s2 and the time instant s3
Program 4: val s4 = Interval[Instant](s2, s3)
Step 5: Find events that are scheduled for this week
Program 5: val s5 = findEvents(createAt(`this`[Interval[Date]] and isWeek))
Step 6: Filter the list of events s5 to keep only those events that intersect with the interval s4
Program 6: val s6 = s5.filter(x => x.interval.intersects(s4))
Step 7: Update each event in the list s6 to happen on the next Friday
Program 7: val s7 = s6.map(x => modifyEvent(x, createAt(next[Date] /& Friday)))
<EOS>

Utterance: Extend pizza party at 1 PM to end at 2.30 pm if extending it doesn't overlap with the next event.
Step 1: Find event called "pizza party"
Program 1: val s1 = theEvent(called("pizza party") and queryAt((1).pm))
Step 2: Find events starting between 1 pm and 2:30 pm that are not titled "pizza party"
Program 2: val s2 = findEvents(not(called("pizza party")) and queryAt(timeInterval(1.pm, (2::30).pm)))
Step 3: If the list s2 is empty then update the event s1 to end at 2:30 pm.
Program 3: val s3 = Option.when(s2.isEmpty){modifyEvent(s1, endsAt((2 :: 30).pm))}
<EOS>

Utterance: Create a preparation meeting this week with the attendees of the project sync who report to me or my manager.
Step 1: Find the event called "project sync"
Program 1: val s1 = theEvent(called("project sync"))
Step 2: Find my reports
Program 2: val s2 = thePerson(me).directReports
Step 3: Find my manager's reports
Program 3: val s3 = thePerson(me).manager.directReports
Step 4: Filter the list of persons s2 to include only people attending the event s1
Program 4: val s4 = s2.filter(x => s1.attendees.isAttending(x))
Step 5: Filter the list of persons s3 to include only the people attending the event s1
Program 5: val s5 = s3.filter(x => s1.attendees.isAttending(x))
Step 6: Create an event called preparation meeting this week with the list of persons s4 and the list of persons s5
Program 6: val s6 = createEvent(with_(s4) and with_(s5) and createAt(thisWeek) and called("preparation meeting"))
<EOS>

Utterance: Other than my manager, how many people are attending the project sync meeting tomorrow?
Step 1: Find the event called project sync tomorrow
Program 1: val s1 = theEvent(called("project sync") and queryAt(tomorrow))
Step 2: Describe the number of attendees of the event s1 excluding the manager
Program 2: val s2 = s1.attendees.all.size - 1
<EOS>

Utterance: Calculate how many meetings last week I had during my lunch hours of 12 noon to 1 PM
Step 1: Find events from last week between 12 PM and 1 PM
Program 1: val s1 = findEvents(queryAt(timeInterval(12.pm, 1.pm) on (last[Interval[Date]] and isWeek)))
Step 2: Describe the number of events in the list s1
Program 2: val s2 = s1.size
<EOS>

Utterance: Find all the meetings scheduled for next week that I created but that conflict with my doctor's appointment. Reschedule them to after the doctor's appointment.
Step 1: Find the event called "doctor's appointment"
Program 1: val s1 = theEvent(called("doctor's appointment"))
Step 2: Find all events happening next week
Program 2: val s2 = findEvents(queryAt(next[Interval[Date]] and isWeek))
Step 3: Filter the list of events s2 to include only the events that are not the event s1
Program 3: val s3 = s2.filter(x => x.id != s1.id)
Step 4: Filter the list of events s3 to include only the events organized by me and which intersect with the event s1
Program 4: val s4 = s3.filter(x => x.organizerIs(me) && x.interval.intersects(s1.interval))
Step 5: Update each event in the list s3 to happen after the event s1
Program 5: val s5 = s4.map(x => modifyEvent(x, startsAt(after(s1.interval))))
<EOS>

Utterance: Update the team meeting on Wednesday, so that its after 8 AM and before 5 PM for Jack
Step 1: Find the event called "team meeting" on Wednesday.
Program 1: val s1 = theEvent(called("team meeting") and queryAt(Wednesday))
Step 2: Find out what time zone Jack is in.
Program 2: val s2 = thePerson(named("Jack")).timeZone
Step 3: Update the event s1 to happen after 8am and before 5pm in the time zone s2.
Program 3: val s3 = modifyEvent(s1, startsAt(after(8.am) inZone s2) and endsAt(before(5.pm) inZone s2))
<EOS>

Utterance: How many 1/1 meetings in total I had in the last week?
Step 1: Find all one on one events from last week and return the size of that list
Program 1: val s1 = findEvents(queryAt(last[Interval[Date]] and isWeek) and isOneOnOne).size
<EOS>

Utterance: If I don't have an email about shiproom, then set up a 1:1 with Smith titled Discussion about Shiproom
Step 1: Check if there are no emails about shiproom
Program 1: val s1 = findEmails(messageTitleIs("shiproom")).isEmpty
Step 2: If s1 is true, set up a 1:1 with Smith titled "Discussion about Shiproom"
Program 2: val s2 = Option.when(s1){createEvent(isOneOnOne and with_("Smith") and called("Discussion about Shiproom"))}
<EOS>

Utterance: Adjust my schedule making sure there are no conflicts with the happy hour event today
Step 1: Find the event called "happy hour" today
Program 1: val s1 = theEvent(called("happy hour") and queryAt(today))
Step 2: Find all events happening today
Program 2: val s2 = findEvents(queryAt(today))
Step 3: Filter the list of events s2 to include only the events that intersect with the event s1
```

Figure A3: Example Prompt to generate program fragment for a generated step. The initial part of the prompt comprises of up to $M \leq 25$ examples similar to *"Filter the list of events s2 to include only the events that intersect with the event s1"*. It is followed by $K = 10$ decompositions of complex utterances. The generated output for the above prompt was `val s3 = s2.filter(x => x.interval.intersects(s1.interval))`

	EXPERTS	DECINT
Grammar	98.3	98.6
Factuality	99.4	99.4
Hallucination	99.4	99.4
Redundancy	99.7	99.7
Repetition	100	100
Missing Information	99.7	97.8
Coherency	98.8	99.7
Commonsense	100	100
Arithmetic	100	100

Table A2: Analysis of the NL steps written by domain experts and those predicted by DECINT. For the overall evaluation, the score is the percentage of examples judged as *sufficient and correct* to complete the user request. For the step-by-step evaluation done by crowd workers, the score of a dimension is the percentage of steps judged as not containing related issues.

no change, suggesting that just a couple of relevant examples are usually enough for the parser.
(4) Reduce train data size to around 20% of the original size: This leads to a reduction in accuracy values, as in-context learning relies on having similar relevant examples in the prompt.

As mentioned earlier, a limitation of the exact match metric is that it is not perfect, since it fails to capture extra redundant braces, or small variations in string argument values, such as the meeting name being "coffee meeting" instead of "coffee". We observe that some of the incorrect predictions are due to a few functions in gold program not being present in the training data (our splits were random). Using API documentation and containing the outputs of the parser (Shin et al., 2021; Roy et al., 2022) to only use allowed functions could be leveraged to fix such errors, though we leave it for future work, as our primary goal is to study complex utterance parsing in light of training data for simpler elementary utterances.

Additional Example Outputs

Figure A4 shows predictions from DECINT and DIRECT-PRED on a few sample inputs. Figure A5 shows predictions from DECINT and the two main baselines DIRECT-PRED and COT on a sample input. Figure A6 shows a few cases where outputs from DECINT were incorrect.

Step-by-Step Evaluation of NL Decomposition: Following ROSCOE (Golovneva et al., 2023),[9] we evaluate the quality of individual steps on 9 dimensions: grammar, factuality, hallucination, redundancy, repetition, missing information, coherency, commonsense, and arithmetic. We recruited annotators from Amazon Mechanical Turk to provide binary classification ratings (*yes* or *no*) for each step in the decomposition on all 9 dimensions. For each question, 3 judgments are collected and the majority-voted answer is used as the final judgment. For quality control, we restrict to annotators located in the United States or Canada who have an approval rate higher than 85% and have successfully solved a qualification task where we match their answers on the same set of questions against answers manually annotated by one of the authors. We pay 0.25 USD per example per question. The step-by-step evaluation results shown in the bottom block of Table A2 suggest that the quality of individual steps is very high. DECINT-predicted steps are rated similar to expert steps on almost all dimensions, except on the "missing information" dimension, where the gap is noticeable. Note that it is possible for all steps in a program to be judged individually correct, but fail to complete the user request.

[9]We adopted ROSCOE as much as possible, though we received some feedback from crowdworkers that some dimensions (such as factuality, hallucination, and coherency) were difficult to judge for our evaluation task.

System	Correct↑	CER↓
DIRECT-PRED	0.31	0.46
DECINT	**0.41**	**0.41**

Table A3: Automated Metrics averaged over 3 runs.

Train and Test Split of Complex Utterances

As noted previously, we use $K = 10$ complex utterance examples from *DeCU* as train split. The examples were chosen randomly. We investigate the impact of random seed, by repeating the experiments for 3 random seeds (i.e. each seed leads to a different set of 10 examples in the train split while the remaining are in the test split), and report the average automated scores in Table A3.

Additional Details on Correct Reference-less Metric using GPT4

Correlation of model-based metrics with human labels: On a per-example basis, we observe that automatic labels and human evaluation labels agree on correct/incorrect annotation 96.5% of the time when compared on the subset of well-formed outputs from the above-mentioned methods. More-

over, considering human-provided labels as ground truth and **Correct** labels as predictions in a binary classification setup, we observe F1 scores of 98% and 84% for “correct" and “incorrect" classes respectively. Thus, **Correct** seems to be a good proxy for a human label to judge the correctness of a generated program.

Prompt: Prompt used in the reference-less evaluation is shown in Figure A7.

Utterance 1: *Make me a meeting at 3 pm mountain time and refuse any meetings that happen at the same time.*

DECINT

```
Step 1: Find events at 3 pm mountain time
val s1 = findEvents(queryAt(3.pm inZone TimeZone("MST")))
Step 2: Refuse any events in the list s1
val s2 = s1.map(x => respond(x, withResponse(ResponseStatusType.declined)))
Step 3: Create a meeting at 3 pm mountain time
val s3 = createEvent(createAt(3.pm inZone TimeZone("MST")))
```

DIRECT-PRED

```
val s1 = createEvent(createAt(3.pm inZone TimeZone("MST")))
val s2 = findEvents(queryAt(3.pm inZone TimeZone("MST")))
val s3 = s2.map(x => refuseEvent(x))
```

Comment We observe that hallucination of functions is much more prominent in the baseline DIRECT-PRED, as can be seen in the use of the non-existing function `refuseEvent`

Utterance 2: *Cross all my vacation events off my calendar, I need to juggle those around.*

DECINT

```
Step 1: Find all events called "vacation"
val s1 = findEvents(called("vacation"))
Step 2: Delete all events in the list s1
val s2 = s1.map(x => deleteEvent(x))
```

DIRECT-PRED

```
val s = theEvent(called("vacation")).map(x => deleteEvent(x))
```

Comment Prediction from DIRECT-PRED incorrectly applies a `map` operation to a single event. We note that predictions from DECINT show fewer syntax errors compared to DIRECT-PRED.

Utterance 3: *If Issac cannot make it to the LT-meeting, schedule some 1:1 time with him tonight.*

DECINT

```
Step 1: Find the event called "LT-meeting"
val s1 = theEvent(called("LT-meeting"))
Step 2: Check if Issac is attending the event s1
val s2 = s1.attendees.isAttending(thePerson("Issac"))
Step 3: If s2 is false, create a 1:1 event with Issac tonight
val s3 = Option.when(!s2){createEvent(isOneOnOne and with_("Isaac") and
    createAt(tonight))}
```

DIRECT-PRED

```
val s1 = theEvent(called("LT-meeting"))
val s2 = thePerson(named("Issac"))
val s3 = s1.attendees.isAttending(s2)
val s4 = if (s3) {
createEvent(with_(s2) and createAt(tonight) and called("1:1 time"))
} else {
null
}
```

Comment DIRECT-PRED output has a logical flaw — it creates a meeting when Issac is attending the 'LT-meeting' – and was judged as incorrect. We observe that outputs from DECINT are less likely to demonstrate such logical flaws.

Utterance 4: *I need you to delete the meeting I have at 3 pm and replace it with "Core" with Jimmy.*

DECINT

```
Step 1: Find the event at 3 pm
val s1 = theEvent(queryAt(3.pm))
Step 2: Delete the event s1
val s2 = deleteEvent(s1)
Step 3: Create a new event called "Core with Jimmy" at 3 pm
val s3 = createEvent(called("Core with Jimmy") and createAt(3.pm))
```

DIRECT-PRED

```
val s1 = deleteEvent(theEvent(queryAt(3.pm)))
val s2 = createEvent(called("Core") and with_("Jimmy"))
```

Comment Prediction from DIRECT-PRED fails to create the meeting at the correct time, and was judged as incorrect in human evaluations.

Utterance 5: *Rename all the 1/1 meetings I have with a PM this month to be called project sync.*

DECINT

```
Step 1: Find all 1/1 meetings this month
val s1 = findEvents(queryAt(`this`[Interval[Date]] and isMonth) and isOneOnOne)
Step 2: Filter the list of events s1 to include only the events with a PM
val s2 = s1.filter( x => getPersonFromAttendee(x.attendees.otherPeople.head).jobTitle
    == Some("PM"))
Step 3: Update each event in the list s2 to be called "project sync"
val s3 = s2.map(x => modifyEvent(x, called("project sync")))
```

DIRECT-PRED

```
val s1 = findEvents(queryAt(`this`[Interval[Date]] and isMonth) and isOneOnOne)
val s2 = s1.filter(x => x.attendees.isAttending(thePerson(PM)))
val s3 = s2.map(x => modifyEvent(x, called("project sync")))
```

Comment DIRECT-PRED output was not valid as per library since 'thePerson(PM)' is incorrect usage while DECINT output was judged as correct.

Figure A4: Outputs from DECINT and DIRECT-PRED on a few complex utterances in *DeCU*.

Utterance: *Make me a meeting at 3 pm mountain time and refuse any meetings that happen at the same time.*

DECINT

```
Step 1: Find events at 3 pm mountain time
val s1 = findEvents(queryAt(3.pm inZone TimeZone("MST")))
Step 2: Refuse any events in the list s1
val s2 = s1.map(x => respond(x, withResponse(ResponseStatusType.declined)))
Step 3: Create a meeting at 3 pm mountain time
val s3 = createEvent(createAt(3.pm inZone TimeZone("MST")))
```

DIRECT-PRED

```
val s1 = createEvent(createAt(3.pm inZone TimeZone("MST")))
val s2 = findEvents(queryAt(3.pm inZone TimeZone("MST")))
val s3 = s2.map(x => refuseEvent(x))
```

COT

```
Step 1: Create a meeting at 3 pm mountain time
Step 2: Refuse any meetings that happen at the same time"}
val s1 = createEvent(createAt(3.pm inZone TimeZone(\"MST\")))
val s2 = findEvents(queryAt(3.pm inZone TimeZone(\"MST\"))).map(x => refuseEvent(x))
```

Figure A5: Outputs from DECINT, DIRECT-PRED and COT on a sample complex utterance in *DeCU*.

Utterance 1: *Split my project sync call next week into two events with the same attendees, but in the second one, invite David Smith also.*

DECINT

```
Step 1: Find the event called "project sync" next week
Plan 1: val s1 = theEvent(called("project sync") and queryAt(next[Interval[Date]] and
    isWeek))
Step 2: Get the attendees of the event s1
Plan 2: val s2 = s1.attendees.all
Step 3: Create a new event with the same attendees as the event s1
Plan 3: val s3 = createEvent(with_(s2) and createAt(s1.interval))
Step 4: Add David Smith to the list of attendees s2
Plan 4: val s4 = s2 :+ thePerson(named("David Smith"))
Step 5: Create a new event with the list of attendees s4
Plan 5: val s5 = createEvent(with_(s4) and createAt(s1.interval))
```

DIRECT-PRED

```
val s1 = theEvent(called("project sync") and queryAt(next[Interval[Date]] and isWeek))
val s2 = createEvent(with_(s1.attendees.all) and createAt(s1.interval.start) and
    called("project sync 1"))
val s3 = createEvent(with_(s1.attendees.all) and with_("David Smith") and
    createAt(s1.interval.end) and called("project sync 2"))
```

Experts

```
Step 1: Find the "project sync" event next week
val s1 = theEvent(called("project sync") and queryAt(next[Interval[Date]] and isWeek))
Step 2: Create an event with the same attendees from event s1 and include David Smith
val s2 = createEvent(with_(s1.attendees) and with_("David Smith"))
```

Comment Output program from DECINT as well as the output from DIRECT-PRED doesn't delete the original meeting and creates two new meetings, which does not reflect the user intention.

Utterance 2: *I'd like to divide the meeting at noon in two, so split off the last fifteen minutes as a separate event.*

DECINT

```
Step 1: Find the event at noon
val s1 = theEvent(createAt(noon))
Step 2: Subtract 15 minutes from the end time of the event s1
val s2 = s1.end - 15.minutes
Step 3: Update the event s1 to end at the time instant s2
val s3 = modifyEvent(s1, endsAt(s2))
Step 4: Create a new event starting at the time instant s2
val s4 = createEvent(startsAt(s2))
```

DIRECT-PRED

```
val s1 = theEvent(createAt(noon))
val s2 = modifyEvent(s1, endsAt(s1.start - 15.minutes))
val s3 = createEvent(startsAt(s1.start - 15.minutes) and lastsFor(15.minutes))
```

Experts

```
Step 1: Find my event at noon
val s1 = theEvent(queryAt(noon))
Step 2: Reduce the duration of event s1 by 15 minutes
val s2 = modifyEvent(s1, lastsFor(s1.duration - 15.minutes))
Step 3: Get the end time of event s2
val s3 = s2.end.local.time
Step 4: Create an event that starts at the end of event s3 that lasts for 15 minutes
val s4 = createEvent(createAt(s3) and lastsFor(15.minutes))
```

Comment Output program from DECINT was judged as incorrect since it doesn't specify the duration of the second event.

Figure A6: Sample predictions from DECINT that were judged as incorrect in human evaluations.

```
Task is to identify whether a program, written in a Scala-like representation, is sufficient and correct to complete the user request.
The program can be considered to be an incorrect solution for various reasons such as if it addresses only a part of user request, or does more than what
    user asked for, or uses incorrect values or variable names, etc.
The program can be considered to be correct if it reflects the user intent, and nothing more, nothing less.


# Utterance: Find all the meetings scheduled for next week that I created but that conflict with my doctor's appointment. Reschedule them to after the
    doctor's appointment.
# Predicted program
val s1 = findEvents(called("doctor")).get(0)
val s2 = findEvents(queryAt(next[Interval[Date]] and isWeek))
val s3 = s2.filter(x => x.organizerIs(me) && x.interval.intersects(s1.interval) && x.id != s1.id)
val s4 = s3.map(x => modifyEvent(x, startsAt(after(s1.end))))
# Judge whether predicted program is correct or not
## Description of predicted program:
s1: Find events about "doctor"
s2: Find events this week
s3: Filter the events in s2 to keep only those that are organized by me and intersect with s1 and are not s1
s4: Modify the events in list s3 to happen after event s1
## Utterance: Find all the meetings scheduled for next week that I created but that conflict with my doctor's appointment. Reschedule them to after the
    doctor's appointment.
## Reflection on predicted program:
In this example, the predicted program correctly reflects the user intent in the utterance.
## Predicted program is: Correct
<EOS>


# Utterance: If I don't have an email about shiproom, then set up a 1:1 with Smith titled Discussion about Shiproom
# Predicted program
val s1 = findEmails(messageTitleIs("shiproom"))
val s2 = e.isEmpty
# Judge whether predicted program is correct or not
## Description of predicted program:
s1: Find the emails about "shiproom"
s2: Check is the list s1 is empty or not
## Utterance: If I don't have an email about shiproom, then set up a 1:1 with Smith titled Discussion about Shiproom
## Reflection on predicted program:
In this example, the predicted program does not create 1:1 meeting as per the user intent from the utterance. Thus, it is not sufficient.
## Predicted program is: Incorrect
<EOS>


# Utterance: Update the team meeting on Wednesday, so that its after 8 AM and before 5 PM for Jack
# Predicted program
val s1 = theEvent(called(\"team meeting\") and queryAt(Wednesday))
val s2 = thePerson(named(\"Jack\")).timeZone
val s3 = modifyEvent(s1, startsAt(after(8.am) inZone s2) and endsAt(before(5.pm) inZone s2))
# Judge whether predicted program is correct or not
## Description of predicted program:
s1: Find the event called "team meeting" on Wednesday.
s2: Find out what time zone Jack is in
s3: Update the event s1 to happen after 8am and before 5pm in the time zone s2.
## Utterance: Update the team meeting on Wednesday, so that its after 8 AM and before 5 PM for Jack
## Reflection on predicted program:
In this example, the predicted program correctly reflects the user intent in the utterance.
## Predicted program is: Correct
<EOS>


# Utterance: Create a preparation meeting this week with the attendees of the project sync who report to me or my manager.
# Predicted program
val s1 = theEvent(called("project sync"))
val s2 = thePerson(me).directReports
val s4 = s2.filter(x => s1.attendees.isAttending(x))
val s6 = createEvent(with_(s4) and createAt(thisWeek) and called("preparation meeting"))
# Judge whether predicted program is correct or not
## Description of predicted program:
s1: Find the event called "project sync"
s2: Find my reports
s4: Filter the list of persons s2 to include only people attending the event s1
s6: Create an event called preparation meeting this week with the list of persons s4
## Utterance: Create a preparation meeting this week with the attendees of the project sync who report to me or my manager.
## Reflection on predicted program:
In this example, the predicted program does not consider the people who report to user's manager.  Thus, it is not sufficient.
## Predicted program is: Incorrect
<EOS>

# Utterance: <test-utterance>
# Predicted Program: <predicted program>
# Judge whether predicted program is correct or not
```

Figure A7: Prompt used to compute Correct metric using GPT4 .

Instructions | Shortcuts

Instructions

In this task, you are asked to rate the quality of a virtual assistant's actions to fulfill a user's request about their calendar.
Please carefully read the instructions below.
You are also strongly encouraged to check some examples by clicking the **"More Instructions"** link at the end of this page.

First, you need to read a relatively complex user utterance in the section **User Request.**
Please try your best to understand the user request.

In the section **Steps**, there is a table describing the virtual assistant's steps for fulfilling the user request.
You are encouraged to read all steps, but please pay attention to the highlighted step in the table.
Steps after the highlighted step are grayed out.
The variable names (s1, s2, ...) in the descriptions refer to the results of the corresponding steps.
For example, *"determine if s1 contains more than 6 meetings"* means *"determine if the result of step 1 (i.e., the list of meetings on Friday) contains more than 6 meetings"*.

The section **Questions** has several required questions about the quality of the highlighted step.
Please choose answers based on your best judgment and provide your confidence about your answers.

If you have any feedback about this task, please enter your response in the **Feedback** section.

Disclaimer

Publications of your responses will be anonymized.

More Instructions

User Request

Move all my one on ones this week to Thursday.

Steps

Step	Description
s1	Find all one on one events this week
s2	Move all events from s1 to Thursday

Questions

Please answer the following questions about the steps in the table above.

Error Type	Question	Answer	Confidence
Grammar	Q1: Does this step contain faulty, unconventional, or controversial grammar usage?	○ Yes ○ No	○ High ○ Medium ○ Low
Factuality	Q2: Does this step contain object or entity (e.g., quantity, date, person) not matching with the user request or preceding steps?	○ Yes ○ No	○ High ○ Medium ○ Low
Hallucination	Q3: Does this step contain irrelevant or wrong information not provided in the user request or preceding steps?	○ Yes ○ No	○ High ○ Medium ○ Low
Redundancy	Q4: Does this step contain redundant information which might be factual but not needed to complete the user request?	○ Yes ○ No	○ High ○ Medium ○ Low
Repetition	Q5: Does this step contain paraphrased information already mentioned in preceding steps?	○ Yes ○ No	○ High ○ Medium ○ Low
Missing Information	Q6: Does this step lack any required information to produce the correct intermediate answer?	○ Yes ○ No	○ High ○ Medium ○ Low
Coherency	Q7: Does this step contradict with the user request or other steps?	○ Yes ○ No	○ High ○ Medium ○ Low
Commonsense	Q8: Does this step lack general knowledge about the world (e.g., number of weekdays in a week)?	○ Yes ○ No	○ High ○ Medium ○ Low
Arithmetic	Q9: Does this step contain errors in math calculations?	○ Yes ○ No	○ High ○ Medium ○ Low

Feedback (Optional)

Figure A8: The annotation interface for the step-by-step evaluation on the NL steps.